Zero to Hero Study Handbook: Repository-Specific SQL & Cypher Query Generator

Module 1: Foundations & Architecture

What this project does

This repository implements a local, tutorial-first pipeline for schema-aware natural language to query generation.

Primary capabilities: - Converts question-style inputs to SQL and Cypher outputs. - Builds deterministic SQL to Cypher labels for dual-task supervision. - Runs prompt-only baselines (`granite4.1:3b`, `qwen3.5:4b`) through Ollama. - Fine-tunes adapters with LoRA/QLoRA (`hf`, `trl`, optional `unsloth` fallback logic). - Evaluates outputs with syntax, grounding, text, latency, optional LLM-judge, and optional Spider execution metrics. - Loads tutorial data into Neo4j for graph-based exploration.

Main use cases from code and docs: - Local experimentation for Text-to-SQL and Text-to-Cypher. - Baseline vs fine-tuned quality comparison. - Reproducible artifact-driven study workflow (`data/processed/*`, `artifacts/*`, `mlruns/*`).

Core paradigms and patterns used here

1. Pipeline orchestration pattern

- Definition: A sequence of explicit stages where each stage emits artifacts consumed by later stages.
- Here: `run_end_to_end(...)` in `src/repo_query_gen/pipeline.py` calls stage runners in a fixed order and writes a manifest.

2. Functional module style with thin CLI wrappers

- Definition: Business logic sits in importable functions; scripts mostly parse args and call one function.
- Here: `scripts/*.py` files call functions like `run_data_preparation`, `run_cypher_extension`, `run_finetuning`.

3. Schema-aware generation

- Definition: Query generation is constrained by schema context and validated against schema references.
- Here: retrieval in `src/repo_query_gen/schema_retrieval.py`, validation in `src/repo_query_gen/inference.py`.

4. Rule-based labeling + optional model refinement

- Definition: Deterministic transformation creates labels; model refinement is optional and gated.
- Here: `sql_to_cypher_deterministic(...)` + `maybe_refine_cypher_with_llm(...)` in `src/repo_query_gen/cypher.py`.

5. Adapter-based fine-tuning with backend resolution

- Definition: Runtime selects an effective trainer backend from requested backend plus availability/compatibility checks.
- Here: `_resolve_backend(...)` in `src/repo_query_gen/training.py`.

6. Artifact-first reproducibility

- Definition: Every stage persists machine-readable outputs for traceability.
- Here: `save_json(...)` in `src/repo_query_gen/utils.py`; stage outputs written under `data/processed/` and `artifacts/`.

Architecture and component interaction

Key components: - Configuration: `src/repo_query_gen/config.py`, `configs/profiles.yaml`, `.env` loading via Pydantic settings. - Data preparation: `src/repo_query_gen/data_prep.py`. - Label extension: `src/repo_query_gen/cypher.py`. - Retrieval + generation: `src/repo_query_gen/schema_retrieval.py`, `baselines.py`, `inference.py`. - Fine-tuning: `src/repo_query_gen/training.py`. - Evaluation: `src/repo_query_gen/evaluation.py`. - Neo4j graph demo: `src/repo_query_gen/neo4j_demo.py`. - Orchestration: `src/repo_query_gen/pipeline.py`.

ASCII main flow:

```
User CLI
  |
  | python scripts/run_end_to_end.py --profile <fast|tutorial|full>
  v
run_end_to_end(...) [src/repo_query_gen/pipeline.py]
  |
  +--> run_data_preparation(...) [data_prep.py]
  |      input: HF dataset Clinton/Text-to-sql-v1
  |      output: data/processed/<profile>/{train,val,test}.csv + manifest.json
  |
  +--> run_cypher_extension(...) [cypher.py]
  |      input: train/val/test.csv
  |      output: train_cypher/val_cypher/test_cypher.csv
  |
  +--> run_baseline_generation(...) [baselines.py]
  |      input: test_cypher.csv
  |      output: artifacts/baseline/<profile>/baseline_predictions.{csv,json}
  |
  +--> (optional) run_finetuning(...) [training.py]
  |      input: train_cypher.csv + val_cypher.csv
  |      output: artifacts/training/<run>/{adapter,train_result,eval_result,...}
  |
  +--> (optional) run_batch_inference(...) [inference.py]
  |      output: artifacts/inference/<profile>/{baseline_granite,baseline_qwen,finetuned}.js
```

```
|
+--> run_evaluation_bundle(...) [evaluation.py]
|      output: artifacts/evaluation/<profile>/{summary_*.json,metrics_per_example.csv,plot
|
+--> (optional) run_neo4j_demo(...) [neo4j_demo.py]
       input: train_cypher.csv
       output: artifacts/neo4j_outputs/<profile>/{graph_summary.json,demo_query_results.js
```

Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`README.md`	End-to-end usage, pipeline stages, command examples	Stage map section, quick-start command sequence	Profile names, Ollama model pull commands
`pyproject.toml`	Dependency and packaging manifest	N/A	`requires-python = ">=3.12.10,<3.13"`, core deps, optional `dev`, optional `unsloth`
`configs/profiles.yaml`	Run-size profiles for fast/tutorial/full	N/A	`dataset_rows`, `max_train_steps`, `eval_sample_size`, `spider_eval_rows`, LoRA hyperparameters
`.env.example`	Optional environment override examples	N/A	`GEN_MODEL_GRANITE`, `GEN_MODEL_QWEN`, `EMBED_MODEL`, `NEO4J_*`, `ENABLE_CYPHER_REFINEMENT`
`docker/docker-compose.neo4j.yml`	Local Neo4j service definition	N/A	`NEO4J_AUTH`, exposed ports `7474/7687`, volumes under `artifacts/neo4j/*`

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
scripts/run_end_to_end.py	Primary runtime CLI entrypoint	main() -> run_end_to_end(...)	--profile, --skip-training, --skip-neo4j, --with-inference, --with-judge, --with-spider, --trainer-backend
scripts/prepare_data.py	Data preparation CLI stage	main() -> run_data_preparation(...)	--profile
scripts/build_cypher_labels.py	SQL-to-Cypher label generation CLI stage	main() -> run_cypher_extension(...)	--profile
scripts/run_baseline.py	Baseline generation CLI stage	main() -> run_baseline_generation(...)	--profile
scripts/train_qlora.py	Fine-tuning CLI stage	main() -> run_finetuning(...)	--profile, --backend, --no-fallback
scripts/infer.py	Single-query inference CLI	main() -> infer_single(...)	--task, --question, --schema-context, --mode, --model-name, --adapter-dir
scripts/evaluate.py	Evaluation CLI stage	main() -> run_evaluation_bundle(...)	--profile, --with-judge, --with-spider
scripts/run_neo4j_demo.py	Neo4j demo CLI stage	main() -> run_neo4j_demo(...)	--profile
src/repo_query_gen/config.py	Runtime settings and profile loading	Settings, ProfileConfig, load_profile(...)	schema_retrieval_mode, training_backend, model names, neo4j_*, seed
src/repo_query_gen/types.py	Typed data contracts	QueryExample, TrainingSample, GeneratedQuery, EvalResult, etc.	Pydantic field names define expected schema
src/repo_query_gen/utils.py	Shared utilities	set_global_seed, save_json, normalize_ws, utc_now_iso	Logging format, deterministic seed behavior

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
src/repo_query_gen/dataset_prep.py	Dataset preprocessing, splitting, and persistence	load_raw_clinton_dataset, build_processed_dataset, stratified_split, run_data_preparation	CREATE_TABLE_RE, COMPLEXITY_PATTERNS
src/repo_query_gen/cypher.py	SQL-to-Cypher conversion and checks	sql_to_cypher_deterministic, validate_cypher_text, run_cypher_extension	enable_cypher_refinement, quality score penalties
src/repo_query_gen/schema_retrieval.py	Schema subset selection	parse_schema_context, select_schema_context	top_k_tables, token overlap scoring
src/repo_query_gen/baselines.py	Prompt-only SQL/Cypher baseline generation	_build_prompt, _generate, run_baseline_generation	ollama_timeout_seconds, schema_retrieval_mode, schema_retrieval_top_k
src/repo_query_gen/training.py	QLoRA training across backends	_resolve_backend, _prepare_datasets, _run_hf_training, _run_trl_training, _run_unsloth_training, run_finetuning	training_backend, allow_backend_fallback, LoRA params from profile
src/repo_query_gen/inference.py	Single and batch inference + validation	infer_single, run_batch_inference, postprocess_generated_query, validate_generated_query	TASK, SQL/Cypher target_query_strategy
src/repo_query_gen/evaluation.py	Metrics, judge scoring, Spider execution benchmark	evaluate_inference_json, evaluate_baseline_csv, run_llm_judging, run_spider_execution_benchmark, run_evaluation_bundle	ENABLE_BERTSCORE, Spider download URLs, metric definitions
src/repo_query_gen/neo4j_demo.py	Neo4j ingestion and demo graph queries	load_dataset_graph, run_demo_queries, run_neo4j_demo	neo4j_uri, neo4j_user, neo4j_password, max_rows=1000
src/repo_query_gen/pipeline.py	Stage orchestration and manifest creation	run_end_to_end, save_manifest, _unload_ollama_models	Stage toggles and training backend flags

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
`tests/*.py`	Regression protection for core logic	Tests for data prep, cypher, retrieval, inference validation, backend resolution	Expected behavior examples for key functions

Recommended first-read subset for new contributors: 1. `README.md` 2. `scripts/run_end_to_end.py` 3. `src/repo_query_gen/pipeline.py` 4. `src/repo_query_gen/config.py` 5. `src/repo_query_gen/data_prep.py` 6. `src/repo_query_gen/inference.py` 7. `src/repo_query_gen/evaluation.py`

Module 3: Core Execution Flows

Flow A: End-to-end orchestration

Entrypoint: - CLI: `scripts/run_end_to_end.py` - Core: `run_end_to_end(...)` in `src/repo_query_gen/pipeline.py`

Core call sequence (from code):

```
manifest["stages"]["data_preparation"] = run_data_preparation(...)
manifest["stages"]["cypher_extension"] = run_cypher_extension(...)
manifest["stages"]["baselines"] = run_baseline_generation(...)
...
manifest["stages"]["evaluation"] = run_evaluation_bundle(...)
```

Key inputs: - `profile_name`: `fast | tutorial | full` - booleans: `include_training`, `include_neo4j`, `include_inference`, `run_judging`, `run_spider_eval` - training control: `training_backend`, `allow_backend_fallback`

Key output shape: - Python dict with top-level keys: `profile`, `started_at`, `stages`, `status`, `finished_at`. - Each stage entry is a dict of artifact paths (serialized to strings by caller scripts).

Flow B: Data preparation (raw dataset -> processed splits)

Entrypoint: - `run_data_preparation(profile_name)` in `src/repo_query_gen/data_prep.py`

Step-by-step: 1. `load_raw_clinton_dataset(...)` - Reads dataset `Clinton/Text-to-sql-v1` with `datasets.load_dataset(..., split="train")`. - Applies profile downsampling when `profile.dataset_rows` is set.

2. `build_processed_dataframe(df)`

- Normalizes SQL text (`normalize_ws`).

- Parses `CREATE TABLE` blocks from `input` field into `tables`, `columns`, and `schema_json`.
- Adds `complexity_tags` via regex patterns (`join`, `group_by`, `nested`, etc.).

3. `stratified_split(...)`

- Stratifies on `source|complexity_bucket`.
- Splits to 80/10/10 using `train_test_split(..., random_state=settings.seed)`.
- Applies optional cap sizes (`train_rows`, `val_rows`, `test_rows`) from profile.

4. `save_processed_splits(...)`

- Writes:
  - `data/processed/<profile>/train.csv`
  - `data/processed/<profile>/val.csv`
  - `data/processed/<profile>/test.csv`
  - `data/processed/<profile>/manifest.json`

Observed processed CSV columns (from `data/processed/tutorial/train.csv` header): - `example_id` - `source` - `instruction` - `question_or_context` - `input_schema_context` - `sql` - `tables` - `columns` - `schema_json` - `complexity_tags` - `complexity_bucket`

Flow C: SQL-to-Cypher extension

Entrypoint: - `run_cypher_extension(profile_name)` in `src/repo_query_gen/cypher.py`

Step-by-step: 1. Reads `train.csv`, `val.csv`, `test.csv`. 2. Parses `complexity_tags` strings back to list-like values. 3. For each row in each split: - `sql_to_cypher_deterministic(sql)` parses SQL with `sqlglot` and builds Cypher-like output. - `maybe_refine_cypher_with_llm(...)` optionally refines only when `settings.enable_cypher_refinement` is true. - `validate_cypher_text(...)` checks presence of `MATCH`, `RETURN`, and bracket balance. - Computes `cypher_quality` score from deterministic and validation signals. 4. Writes: - `train_cypher.csv`, `val_cypher.csv`, `test_cypher.csv`

Added output columns (from `train_cypher.csv` header): - `cypher` - `cypher_status` - `cypher_quality` - `cypher_issues`

Flow D: Retrieval-aware baseline generation and inference

D1. Prompt-only baselines Entrypoint: - `run_baseline_generation(profile_name)` in `src/repo_query_gen/baselines.py`

Step-by-step: 1. Reads `test_cypher.csv`. 2. Samples `min(profile.eval_sample_size, len(test_df))` rows. 3. For each model in `[settings.granite_model, settings.qwen_model]` and each sampled row: - Optionally retrieves schema subset via `select_schema_context(...)` (lexical mode). - Builds SQL prompt and Cypher prompt with `_build_prompt(...)`. - Calls Ollama through

`_generate(...)` with retry/backoff settings. - Records per-task latency. 4. Writes: - `artifacts/baseline/<profile>/baseline_predictions.csv` - `artifacts/baseline/<profile>/baseline_predictions.json`

Observed baseline CSV columns: - `example_id`, `model_name`, `question_or_context` - `schema_context`, `used_schema_context` - `sql_reference`, `sql_pred`, `sql_latency_ms` - `cypher_reference`, `cypher_pred`, `cypher_latency_ms` - `retrieval_strategy`, `retrieval_selected_tables`, `retrieval_selected_columns` - `source`, `complexity_tags`

D2. Single and batch inference Entrypoints: - `infer_single(...)` in `src/repo_query_gen/inference.py` - `run_batch_inference(...)` in `src/repo_query_gen/inference.py`

Critical steps in `infer_single(...)`: 1. Selects schema context (`_select_schema_for_prompt`) using `schema_retrieval_mode`. 2. Generates query with either: - `generate_with_ollama(...)`, or - `generate_with_finetuned(...)` (requires adapter dir). 3. Post-processes output to query-only text: - SQL: `_extract_sql_statement(...)` - Cypher: `_extract_cypher_statement(...)` 4. Validates query:

```
return {
    "parse_success": parse_ok,
    "schema_grounded": schema_ok,
    "issues": issues,
}
```

5. Returns a structured dict with:

- generation fields: `task`, `model`, `question`, `generated_query`, `latency_ms`
- schema fields: `schema_context`, `used_schema_context`, `retrieval`, `schema_references`
- quality fields: `validation`, `explanation`

`run_batch_inference(...)` adds dataset metadata per record: - `example_id`, `source`, `sql_reference`, `cypher_reference`, `question_or_context`

Flow E: Fine-tuning

Entrypoint: - `run_finetuning(profile_name, backend, allow_fallback)` in `src/repo_query_gen/training.py`

Step-by-step: 1. Resolves backend using `_resolve_backend(...)`. - Rules include module-availability checks and `unsloth` compatibility gates.

2. Prepares train/val datasets from `train_cypher.csv` and `val_cypher.csv`.

- `_to_instruction_rows(...)` creates one SQL sample and (if present) one Cypher sample per original row.
- Instruction row shape:

  - `text`
  - `prompt`
  - `completion`
  - `task`
  - `example_id`

3. Builds quantized model + tokenizer (`_build_model_and_tokenizer`).

- Requires CUDA GPU (`RuntimeError` if unavailable).
- Attempts 4-bit load first, with 8-bit CPU-offload fallback.

4. Trains via selected backend (`hf`, `trl`, or `unsloth`) and saves adapter artifacts.

5. Logs metadata/metrics:

- MLflow params and metrics.
- `gpu_snapshot.json`, `train_result.json`, `eval_result.json`, `training_metadata.json`.

Returned artifact map keys: - `run_dir`, `adapter_dir`, `train_result`, `eval_result`, `metadata`

Flow F: Evaluation bundle

Entrypoint: - `run_evaluation_bundle(profile_name, run_judging=False, run_spider=False)` in `src/repo_query_gen/evaluation.py`

Step-by-step: 1. Checks inference JSON files in `artifacts/inference/<profile>/` for: - `baseline_granite.json` - `baseline_qwen.json` - `finetuned.json`

2. If baseline inference JSON is missing, falls back to `artifacts/baseline/<profile>/baseline_predictio`

3. Computes metrics using:

- `exact_match`, `sql_parse_success`, `cypher_parse_success`
- `schema_grounding_accuracy`
- `text_metrics` (`bleu`, `rouge_l`, `meteor`, optional `bertscore_f1`)
- retrieval metric: `_retrieval_table_recall`
- latency percentiles: `latency_ms_p50`, `latency_ms_p95`

4. Optionally runs:

- LLM judge (`run_llm_judging`) with both configured local models.
- Spider execution benchmark (`run_spider_execution_benchmark`).

5. Writes summaries and combined outputs:

- `summary_<label>.json`
- `metrics_per_example.csv`
- plot images under `plots/`

Observed summary JSON keys (`artifacts/evaluation/tutorial/summary_finetuned.json`): - `exact_match` - `syntax_success` - `schema_grounding` - `bleu` - `rouge_l` -

meteor - bertscore_f1 - retrieval_table_recall - latency_ms_p50 - latency_ms_p95

Flow G: Neo4j graph demo

Entrypoint: - `run_neo4j_demo(profile_name)` in `src/repo_query_gen/neo4j_demo.py`

Step-by-step: 1. Reads `data/processed/<profile>/train_cypher.csv`, takes `head(max_rows)` (default 1000). 2. Waits for Neo4j (`wait_for_neo4j`) and clears graph (`reset_graph`: `MATCH (n) DETACH DELETE n`). 3. MERGEs graph entities: - Nodes: `Source`, `Question`, `SqlQuery`, `CypherQuery`, `Table` - Relationships: `FROM_SOURCE`, `HAS_SQL`, `HAS_CYPHER`, `USES_TABLE`, `MATCHES_TABLE` 4. Runs demo Cypher queries (`top_sources`, `most_used_tables`, `questions_with_many_tables`). 5. Writes: - `graph_summary.json` - `demo_query_results.json`

Module 4: Setup & Run Guide

1. Clean machine prerequisites

From repository files: - Python: `.python-version` is `3.12.10`. - Package manager: `uv` workflow is defined in `README.md` and lockfile `uv.lock`. - Container runtime: required for Neo4j (`docker/docker-compose.neo4j.yml`). - Local model runtime: Ollama is used by baselines, inference, and judge paths (`baselines.py`, `inference.py`, `evaluation.py`).

2. Environment and dependency install

Recommended sequence (from `README.md`):

```
cd /home/ahmad/AI/Github/Repository-Specific-SQL-Cypher-Query-Generator
UV_CACHE_DIR=/tmp/uv-cache uv venv --python 3.12.10
source .venv/bin/activate
UV_CACHE_DIR=/tmp/uv-cache uv sync
```

3. Optional `.env` configuration

`.env` is loaded by `Settings` via `env_file=".env"` in `src/repo_query_gen/config.py`.

Keys present in `.env.example`: - `GEN_MODEL_GRANITE` - `GEN_MODEL_QWEN` - `EMBED_MODEL` - `HF_HOME` - `NEO4J_URI` - `NEO4J_USER` - `NEO4J_PASSWORD` - `ENABLE_CYPHER_REFINEMENT`

Important note from static code reading: - `Settings` fields are named `granite_model`, `qwen_model`, `embed_model`, etc. - Without explicit aliases in `config.py`, the direct environment names expected by Pydantic settings typically follow field names (for example `GRANITE_MODEL`, `QWEN_MODEL`, `EMBED_MODEL`). - So `GEN_MODEL_GRANITE` and `GEN_MODEL_QWEN` in `.env.example` may not override `Settings` unless mapped elsewhere (no mapping exists in this repository code).

4. External services and model assets

Neo4j service (from `README.md` and `docker/docker-compose.neo4j.yml`):

```
docker compose -f docker/docker-compose.neo4j.yml up -d
```

Ollama models (from `README.md`):

```
ollama pull granite4.1:3b
ollama pull qwen3.5:4b
ollama pull qwen3-embedding:4b
```

5. Typical command sequences

Single command for full pipeline orchestration:

```
python scripts/run_end_to_end.py --profile fast --trainer-backend auto
```

Tutorial profile with extra evaluation paths:

```
python scripts/run_end_to_end.py \
  --profile tutorial \
  --with-inference \
  --with-judge \
  --with-spider \
  --trainer-backend auto
```

Stage-by-stage commands:

```
python scripts/prepare_data.py --profile tutorial
python scripts/build_cypher_labels.py --profile tutorial
python scripts/run_baselines.py --profile tutorial
python scripts/train_qlora.py --profile tutorial --backend auto
python scripts/evaluate.py --profile tutorial --with-judge --with-spider
python scripts/run_neo4j_demo.py --profile tutorial
```

Single inference example:

```
python scripts/infer.py \
  --task sql \
  --mode ollama \
  --question "How many countries exist?" \
  --schema-context "CREATE TABLE countries (id INT, name TEXT);"
```

6. Migration or seeding steps

Database migrations: - No migration framework exists in this repository.

Data seeding equivalent: - Relational training/eval data is produced by `scripts/prepare_data.py` and `scripts/build_cypher_labels.py`. - Neo4j graph data is loaded by `scripts/run_neo4j_demo.py` (which resets the graph before loading).

External benchmark assets: - Spider benchmark data is downloaded on demand by `_download_spider_dataset(...)` in `src/repo_query_gen/evaluation.py` when `--with-spider` is enabled.

Module 5: Study Plan & Practice Exercises

Ordered self-study path

1. Orientation and command surface

- Read: `README.md`, then all files in `scripts/`.
- Goal: know every stage command and CLI flag.

2. Configuration system

- Read: `src/repo_query_gen/config.py`, `configs/profiles.yaml`, `.env.example`.
- Goal: understand what is profile-driven vs environment-driven.

3. Data contracts and utilities

- Read: `src/repo_query_gen/types.py`, `src/repo_query_gen/utils.py`.
- Goal: internal object shapes and reproducibility helpers.

4. Data and labeling pipeline

- Read: `src/repo_query_gen/data_prep.py`, `src/repo_query_gen/cypher.py`.
- Goal: how raw rows become SQL/Cypher supervised data.

5. Retrieval + generation path

- Read: `src/repo_query_gen/schema_retrieval.py`, `src/repo_query_gen/baselines.py`, `src/repo_query_gen/inference.py`.
- Goal: prompt construction, retrieval strategy, and output validation.

6. Training internals

- Read: `src/repo_query_gen/training.py`.
- Goal: backend resolution, LoRA attachment, and artifact logging.

7. Evaluation internals

- Read: `src/repo_query_gen/evaluation.py`.
- Goal: metric definitions, judge path, and Spider execution path.

8. Graph demo and orchestration

- Read: `src/repo_query_gen/neo4j_demo.py`, `src/repo_query_gen/pipeline.py`.
- Goal: full system integration and stage manifest understanding.

9. Tests as behavior specification

- Read: `tests/test_*.py`.
- Goal: expected behavior for retrieval, inference parsing, and backend fallback.

Practice exercises (with solution outlines)

Exercise 1 Question: Which exact profile keys control fine-tuning runtime length and LoRA capacity?

Where to look: - `configs/profiles.yaml` - `src/repo_query_gen/training.py`

Solution outline: - Runtime length: `max_train_steps`, `batch_size`, `gradient_accumulation_steps`, `max_seq_len`. - LoRA capacity: `lora_r`, `lora_alpha`, `lora_dropout`. - These values are read through `load_profile(...)` and passed to training config/builders.

Exercise 2 Question: How is `example_id` created, and where does it travel in later stages?

Where to look: - `src/repo_query_gen/data_prep.py` - `src/repo_query_gen/inference.py` - `src/repo_query_gen/evaluation.py`

Solution outline: - Created in `build_processed_dataframe(...)` as `clinton_{idx}`. - Preserved in processed CSVs. - Added to each inference output row in `run_batch_inference(...)`. - Used in evaluation records (`evaluate_inference_json`).

Exercise 3 Question: Why can `train_cypher.csv` have fewer rows than `train.csv`?

Where to look: - `src/repo_query_gen/cypher.py` - README processed-data note

Solution outline: - `build_cypher_labels_for_split(...)` drops rows where `sql` is invalid/empty (`invalid_mask`). - So Cypher split can be smaller than SQL split.

Exercise 4 Question: Explain exactly how lexical schema retrieval scores tables.

Where to look: - `src/repo_query_gen/schema_retrieval.py`

Solution outline: - Tokens are extracted from question, table name, and column names. - Score = `3 * overlap(question, table_name) + 1 * overlap(question, columns)`. - Top tables with score > 0 are kept; if none score > 0, first `top_k_tables` are selected.

Exercise 5 Question: How does the system remove non-query text from model outputs before evaluation?

Where to look: - `src/repo_query_gen/inference.py` - `tests/test_inference_validation.py`

Solution outline: - `_strip_fenced_block(...)` extracts code fences when present. - SQL path uses `_extract_sql_statement(...)` with parser checks

and semicolon chunking. - Cypher path uses `_extract_cypher_statement(...)` and removes trailing explanation sections. - `postprocess_generated_query(...)` is the public normalization function.

Exercise 6 Question: What happens if `--backend unsloth` is requested but model compatibility fails?

Where to look: - `src/repo_query_gen/training.py` - `tests/test_training_backend.py`

Solution outline: - `_resolve_backend(...)` checks module availability and `_unsloth_model_supported(...)`. - If incompatible and fallback allowed, it falls back to `trl` (if installed) else `hf`. - Test `test_unsloth_incompatible_model_falls_back_to_trl` asserts this behavior.

Exercise 7 Question: If baseline inference JSON does not exist, how does evaluation still proceed?

Where to look: - `src/repo_query_gen/evaluation.py`

Solution outline: - `run_evaluation_bundle(...)` checks for inference JSON files first. - For baseline labels, if missing, it reads `artifacts/baseline/<profile>/baseline_predictions.cs` - It then filters by target model (`settings.granite_model` or `settings.qwen_model`) and computes summary.

Exercise 8 Question: Describe the graph schema loaded into Neo4j and one business question each relation can answer.

Where to look: - `src/repo_query_gen/neo4j_demo.py`

Solution outline: - Nodes: `Source`, `Question`, `SqlQuery`, `CypherQuery`, `Table`. - Edges: - `Question-[:FROM_SOURCE]->Source`: "Which source contributes most questions?" - `Question-[:HAS_SQL]->SqlQuery`: "What SQL corresponds to a question?" - `Question-[:HAS_CYPHER]->CypherQuery`: "What Cypher label was generated?" - `SqlQuery-[:USES_TABLE]->Table`: "Which tables are most used in SQL?" - `CypherQuery-[:MATCHES_TABLE]->Table`: "Which table entities appear in Cypher labels?"

Verification Checklist

Use this checklist to confirm mastery:

- Can you explain `run_end_to_end(...)` stage order and optional flags without opening code?
- Can you trace one row from raw dataset to `train.csv` to `train_cypher.csv` to inference/evaluation artifacts?
- Can you describe how lexical retrieval selects schema context and why `top_k_tables` matters?

- Can you explain exactly how `infer_single(...)` builds, postprocesses, and validates a query?
- Can you describe backend resolution rules for `auto`, `trl`, and `unsloth` requests?
- Can you list the metrics emitted in `summary_*.json` and what each one measures?
- Can you explain what `run_neo4j_demo(...)` loads and which graph queries it runs?
- Can you identify the likely `.env` key mismatch risk between `.env.example` and `Settings` field names?
- Can you locate all generated artifacts for one profile and map each to its producing function?